

1. Identificação

Aluno: Lucas Hufnagel Gromann de Araujo Góes

Matrícula: 2410845

Nome: Caio Faria Brito Martins Chaves

Matrícula:2410162

2. Objetivo

O objetivo deste trabalho é explorar o uso de memória compartilhada em sistemas operacionais, utilizando a linguagem C em ambiente Linux.

Os programas desenvolvidos têm como finalidade demonstrar:

A criação e manipulação de memória compartilhada com shmget e shmat;

A comunicação entre processos criados com fork();

Problemas de concorrência, como condições de corrida (race condition);

Impacto da execução paralela na consistência dos dados.

3. Estrutura do Programa

- O programa 1 é composto por:
 - Criação da memória compartilhada:
 - shmget() cria um espaço para armazenar um inteiro.
 - shmat() associa a memória ao ponteiro n.
 - Inicialização:
 - A variável compartilhada n recebe valor inicial 1.
 - Criação de processos filhos:
 - Três processos são criados com fork().
 - Cada processo executa uma operação diferente:
 - Filho 1: soma 1, mil vezes;
 - Filho 2: soma 10, mil vezes;
 - Filho 3: soma 100, mil vezes.
 - Sincronização:
 - O processo pai usa wait() para aguardar os filhos.
 - Finalização:

- Impressão do valor final de n ;
 - Liberação da memória com `shmdt()` e `shmctl()`.
- O programa 2 possui:
 - Criação de memória compartilhada:
 - Um vetor de tamanho 10000 é criado.
 - Inicialização do vetor:
 - Todas as posições recebem valor 1.
 - Criação de 10 processos filhos.
 - Cada processo percorre o vetor e aplica:
 - $\text{vetor}[j] = \text{vetor}[j] * 2 + 2$;
 - Sincronização:
 - O processo pai espera todos os filhos com `wait()`.
 - Validação dos dados:
 - O programa verifica se todos os valores do vetor são iguais.
 - Conta inconsistências.
 - Saída:
 - Exibe o valor final ou erro caso haja divergências.

4. Solução

Programa 1 – Funcionamento passo a passo

- O programa cria uma variável compartilhada n com valor inicial igual a 1.
- Em seguida, são criados três processos filhos.
- Cada filho modifica o valor de n :
 - Filho 1 adiciona 1 repetidamente (1000 vezes);
 - Filho 2 adiciona 10 repetidamente;
 - Filho 3 adiciona 100 repetidamente.
- Como não há controle de concorrência, os processos acessam a variável ao mesmo tempo.
- Isso gera uma condição de corrida, pois múltiplos processos leem e escrevem simultaneamente.
- O valor final pode variar a cada execução.

Saída esperada (exemplo):

Filho 1 (pid=1234), n=...

Filho 2 (pid=1235), n=...

Filho 3 (pid=1236), n=...

Valor final de n = ...

O valor correto esperado (sem concorrência) seria:

$$n = 1 + (1000 \cdot 1) + (1000 \cdot 10) + (1000 \cdot 100)$$

$$n = 111001$$

Porém, devido à concorrência, o valor geralmente será menor ou inconsistente.

Programa 2 – Funcionamento passo a passo

- O programa cria um vetor compartilhado de tamanho 10000.
- Inicializa todas as posições com valor 1.
- São criados 10 processos filhos.
- Cada processo modifica o vetor inteiro aplicando a fórmula:
 - $\text{vetor}[j] = \text{vetor}[j] \cdot 2 + 2;$
- Como todos os processos acessam o mesmo vetor simultaneamente, ocorrem conflitos de escrita.
- Após a execução, o processo pai verifica se todos os elementos possuem o mesmo valor.

Saída possível:

->Caso consistente:

- Todas as posições tem o valor: X

->Caso inconsistente:

- Erro: Encontradas X inconsistências no vetor!

Na prática, devido à ausência de sincronização, é comum ocorrerem inconsistências.

5. Observações e Conclusões

- Ambos os programas demonstram claramente o problema de condições de corrida.
- A principal dificuldade encontrada é a falta de controle de acesso à memória compartilhada.
- Sem mecanismos como:
 - semáforos
 - mutex
 - ou outras formas de sincronização
- os dados tornam-se inconsistentes.
- O primeiro programa mostra inconsistência em uma variável simples, enquanto o segundo amplia o problema para um vetor grande.
- Foi possível observar que:
 - O resultado varia a cada execução;
 - O comportamento é não determinístico;
 - A concorrência sem controle compromete a integridade dos dados.

Conclusão:

O uso de memória compartilhada exige mecanismos de sincronização para garantir consistência. Sem isso, programas concorrentes podem produzir resultados incorretos, mesmo que a lógica esteja aparentemente correta.